

Smart Lender: AI-Powered Loan Eligibility Prediction System

Project Description

Smart Lender is a Machine Learning-powered web application developed to predict the creditworthiness of loan applicants and assist banks and financial institutions in making faster, accurate, and data-driven loan approval decisions. The application uses multiple supervised machine learning classification algorithms, namely **Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost**, to analyze applicant information and determine whether a loan application should be approved or rejected.

The system processes structured applicant data, including **Gender, Marital Status, Number of Dependents, Education, Self-Employment Status, Applicant Income, Co-applicant Income, Loan Amount, Loan Amount Term, Credit History, and Property Area**. These features are preprocessed and transformed before being passed to the trained machine learning models for prediction. After evaluating all four classification algorithms based on accuracy and performance metrics, the **XGBoost classifier** achieved the highest performance with **94.7% training accuracy** and **81.1% testing accuracy**, making it the final model selected for deployment.

The trained XGBoost model is integrated into a **Flask-based web application**, allowing users to enter loan applicant details through a simple and responsive interface. Once the required information is submitted, the application performs preprocessing, loads the trained model, and instantly predicts whether the applicant is eligible for loan approval. The system provides quick and reliable predictions, helping financial institutions minimize manual effort and improve operational efficiency.

Designed for deployment on **IBM Cloud**, Smart Lender offers a scalable, secure, and user-friendly solution that can be accessed through a web browser. The platform enables banks, credit officers, financial analysts, and lending institutions to automate the loan evaluation process, reduce non-performing assets (NPAs), minimize human errors, and make consistent lending decisions based on historical data and machine learning insights.

Scenarios

Scenario 1: Fast-Track Approval for Low-Risk Applicants

A bank loan officer enters the details of a salaried applicant with stable employment, a strong credit history, and sufficient income. The Smart Lender system predicts that the applicant has a high probability of repaying the loan, enabling the bank to approve the application quickly without extensive manual verification.

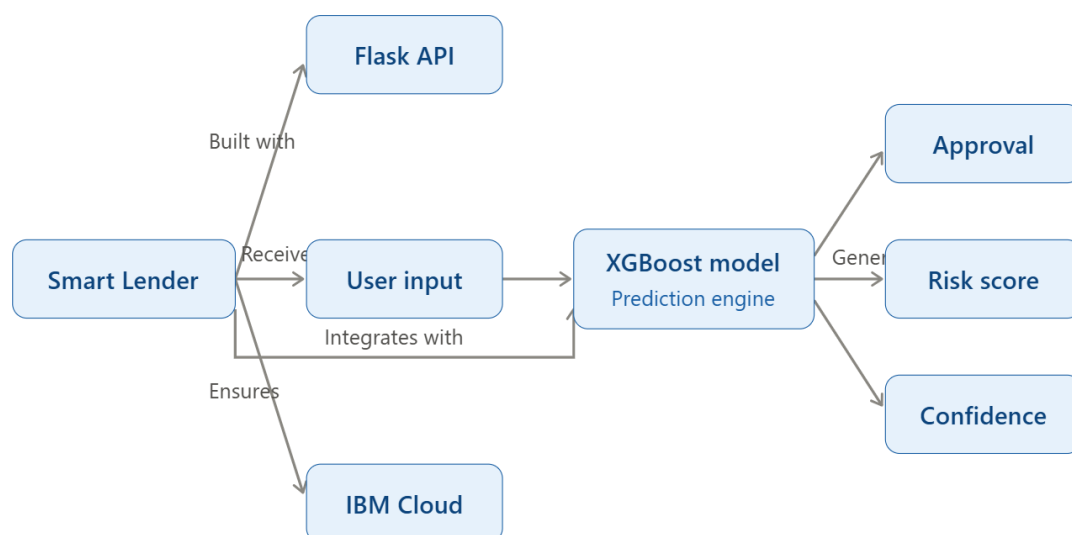
Scenario 2: High-Risk Applicant Detection

An applicant with irregular income, poor repayment history, and no previous credit record applies for a loan. The XGBoost model classifies the application as high-risk and recommends additional document verification before approval, helping the bank reduce the chances of loan default.

Scenario 3: Bulk Loan Application Evaluation

During periods of high loan demand, a financial analyst uploads and evaluates multiple loan applications using Smart Lender. The system predicts loan eligibility for each applicant within seconds, significantly reducing evaluation time while maintaining high prediction accuracy and consistency.

Technical Architecture



Description:

Smart Lender utilizes a modular **three-tier architecture** to provide accurate and real-time loan eligibility prediction. The application consists of a responsive **frontend**, a **Flask-based backend**, and a **Machine Learning prediction layer**.

The frontend is developed using **HTML, CSS, and JavaScript**, providing an intuitive interface where users can enter applicant information such as gender, marital status, education, employment status, applicant income, co-applicant income, loan amount, loan amount

term, credit history, and property area. The interface validates the entered data and securely submits it to the backend for processing.

The backend is built using the **Flask framework**, which manages request handling, input validation, data preprocessing, feature transformation, and communication with the trained machine learning model. It loads the serialized **XGBoost model**, processes the applicant information, and generates a loan approval prediction based on the applicant's credit profile.

The machine learning layer is developed using **Scikit-learn** and **XGBoost**. Multiple classification algorithms, including **Decision Tree**, **Random Forest**, **K-Nearest Neighbors (KNN)**, and **XGBoost**, were trained and evaluated using historical loan applicant data. Among these, **XGBoost** achieved the highest prediction performance with **94.7% training accuracy** and **81.1% testing accuracy**, making it the final model integrated into the application.

The prediction results are returned to the Flask backend and displayed instantly on the web interface, enabling banks and financial institutions to make faster, consistent, and data-driven loan approval decisions. The application is designed for deployment on **IBM Cloud**, providing a secure, scalable, and reliable environment for real-time loan eligibility prediction.

Architecture Components

- **Presentation Layer:** HTML, CSS, JavaScript
- **Application Layer:** Python, Flask Framework
- **Machine Learning Layer:** XGBoost, Decision Tree, Random Forest, KNN
- **Data Processing Layer:** Pandas, NumPy, Scikit-learn
- **Deployment Platform:** IBM Cloud

Project Workflow

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Collect the loan applicant dataset and analyze its features, including applicant income, loan amount, credit history, education, employment status, and property area.
 - **Activity 1.2:** Perform data preprocessing by handling missing values, encoding categorical variables, normalizing numerical features, and splitting the dataset into training and testing sets.
 - **Activity 1.3:** Research and compare multiple machine learning classification algorithms such as Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost to identify the most suitable model for loan eligibility prediction.
 - **Activity 1.4:** Design the application architecture by defining the interaction between the user interface, Flask backend, machine learning model, and prediction module. Configure the Python development environment by installing the required libraries and project dependencies.
-

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop and train the machine learning models using the prepared dataset. Evaluate each model using performance metrics such as accuracy, precision, recall, and F1-score, and select the best-performing model (XGBoost) for deployment.
 - **Activity 2.2:** Implement the Flask backend to process user requests, validate applicant information, perform feature preprocessing, load the trained machine learning model, and generate real-time loan eligibility predictions.
-

Milestone 3: App.py Development

- **Activity 3.1:** Develop the main application logic in **app.py** by defining routes for the home page and prediction page, loading the trained XGBoost model, preprocessing user inputs, performing predictions, and displaying the results through the Flask application.
-

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop a responsive user interface using HTML, CSS, and JavaScript that allows users to enter loan applicant details through an interactive prediction form.
 - **Activity 4.2:** Integrate the frontend with the Flask backend to send applicant information, receive prediction results, and dynamically display whether the loan application is approved or rejected.
-

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for deployment by configuring the Flask server, organizing project files, installing project dependencies, and testing the complete application in the local development environment.
 - **Activity 5.2:** Deploy the Smart Lender application on **IBM Cloud** to provide secure, scalable, and real-time access for financial institutions and end users.
-

Milestone 6: Conclusion

Evaluate the overall performance of the Smart Lender application by validating prediction accuracy, testing the deployed web application, and ensuring that the system provides reliable, fast, and consistent loan approval predictions. The project demonstrates how machine learning and web technologies can be combined to improve decision-making in the banking and financial sector.

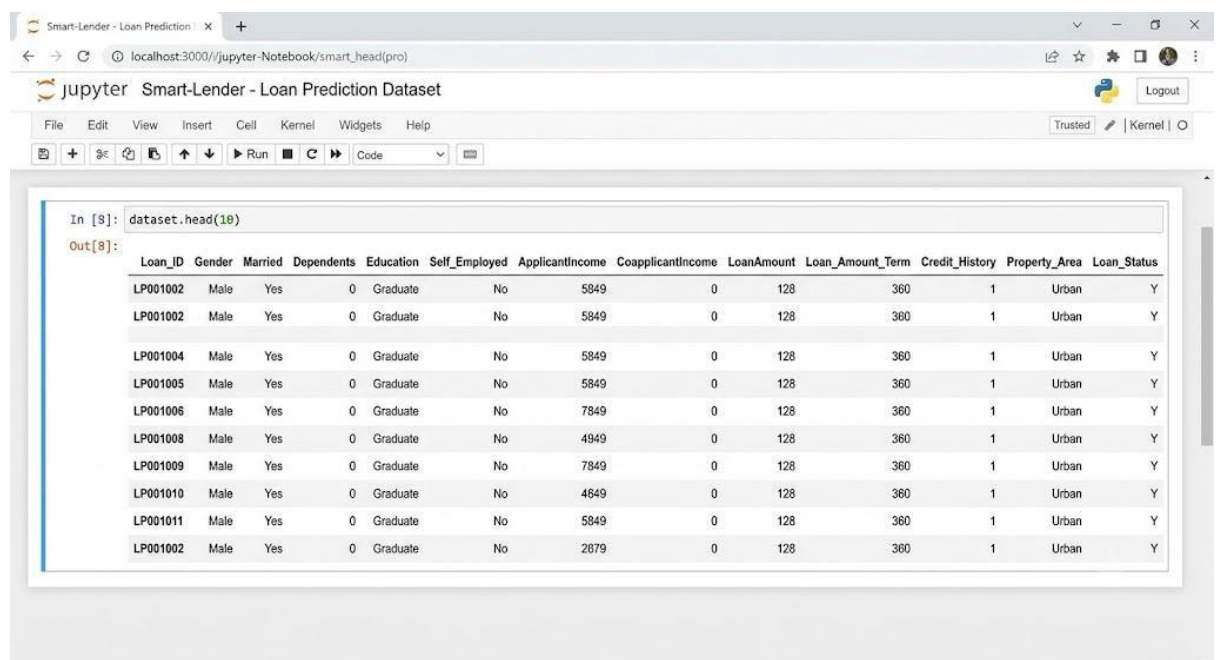
Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the most suitable machine learning classification algorithm for predicting loan eligibility. This involves collecting and preprocessing the loan applicant dataset, analyzing different classification models, evaluating their performance, and selecting the best-performing model for integration into the Smart Lender web application. The application architecture is also designed by defining the interaction between the frontend, Flask backend, and the trained machine learning model.

Activity 1.1: Dataset Collection and Preparation

Before training the machine learning models, a suitable loan applicant dataset is required. Follow the steps below to prepare the dataset for model training.

- **Step 1 – Collect the Dataset:** Obtain a loan prediction dataset containing applicant details such as Gender, Marital Status, Dependents, Education, Self-Employment Status, Applicant Income, Co-applicant Income, Loan Amount, Loan Amount Term, Credit History, Property Area, and Loan Status.



Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
LP001002	Male	Yes	0	Graduate	No	5849	0	128	360	1	Urban	Y
LP001002	Male	Yes	0	Graduate	No	5849	0	128	360	1	Urban	Y
LP001004	Male	Yes	0	Graduate	No	5849	0	128	360	1	Urban	Y
LP001005	Male	Yes	0	Graduate	No	5849	0	128	360	1	Urban	Y
LP001006	Male	Yes	0	Graduate	No	7849	0	128	360	1	Urban	Y
LP001008	Male	Yes	0	Graduate	No	4949	0	128	360	1	Urban	Y
LP001009	Male	Yes	0	Graduate	No	7849	0	128	360	1	Urban	Y
LP001010	Male	Yes	0	Graduate	No	4649	0	128	360	1	Urban	Y
LP001011	Male	Yes	0	Graduate	No	5849	0	128	360	1	Urban	Y
LP001002	Male	Yes	0	Graduate	No	2679	0	128	360	1	Urban	Y

- **Step 2 – Import the Dataset:** Load the dataset into Python using the Pandas library.

import pandas as pd

```
dataset = pd.read_csv("loan_prediction.csv")
```

```

app.py
1 import pandas as pd
2
3 # Define the dataset path
4 data_path = 'loan_prediction.csv'
5
6 # Load the dataset
7 dataset = pd.read_csv(data_path)
8
9 # Display the first five rows
10 print(dataset.head())

```

	Loan_ID	Gender	Married	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
0	LP0001002	Male	Yes	Graduate	No	5849	0	128	360	1	Urban	Y
1	LP0001004	Male	Yes	Graduate	No	5849	0	128	360	1	Urban	Y
2	LP0001005	Male	Yes	Graduate	No	7849	0	128	360	1	Urban	Y
3	LP0001008	Male	Yes	Graduate	No	4949	0	128	360	1	Urban	Y
4	LP0001009	Male	Yes	Graduate	No	4649	0	128	360	1	Urban	Y
5	LP0001011	Male	Yes	Graduate	No	2679	0	128	360	1	Urban	Y

- **Step 3 – Explore the Dataset:** Examine the dataset structure, identify missing values, verify data types, and understand the distribution of each feature.

```

In [1]: dataset.info()

Out[2]:
<class 'Loan Prediction Dataset'>
RangeIndex: 614 entries, 14 com64
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  ---                ---
0   Loan_ID                614 non-null    object
1   Gender                 614 non-null    object
2   Married                614 non-null    object
3   ApplicantIncome         614 non-null    int64
4   CoapplicantIncome       614 non-null    int64
5   LoanAmount              128 non-null    int64
6   Loan_Amount_Term        360 non-null    int64
7   Credit_History          564 non-null    object
8   Credit_Status           614 non-null    object
9   Property_Area           614 non-null    object
dtypes: float64, object()

In [2]: dataset.describe()

Out[2]:

```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	614.000000	614.000000	614.000000	614.000000	614.000000
mean	5849.1600	60.293225	128.6128	360	1.633233
std	13.286255	11.659795	21.73827	360	1.381325
min	0.00	0.00	0.00	360	1
25%	5849.00	0.00	128.00	360	1
50%	7849.00	0.00	128.00	360	1
75%	4649.00	0.00	128.00	360	1
max	200000.00	2679.00	128.00	360	1

Output of: `dataset.head()`
`dataset.info()`
`dataset.describe()`

- **Step 4 – Handle Missing Values:** Replace or remove missing values using appropriate preprocessing techniques to improve model performance.

- **Step 5 – Encode Categorical Features:** Convert categorical attributes such as Gender, Education, and Property Area into numerical values using Label Encoding or One-Hot Encoding.
- **Step 6 – Split the Dataset:** Divide the dataset into training and testing datasets using Scikit-learn.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

Activity 1.2: Research and Select the Appropriate Machine Learning Model

The following activities were performed to identify the best classification model for loan eligibility prediction.

- **Understand the Project Requirements:** Analyze the objective of predicting loan approval based on applicant financial and personal information.
- **Research Classification Algorithms:** Study various supervised machine learning algorithms including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost.
- **Evaluate Model Performance:** Train and compare each model using evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.
- **Select the Best Model:** Choose the **XGBoost Classifier** because it achieved the highest prediction performance with **94.7% training accuracy** and **81.1% testing accuracy**, making it the most suitable model for deployment.

```

File Edit Selection View Go Run Terminal Help
app.py - Jupyter Notebook
File Edit View Insert Cell Kernel Widgets Help
soonh Kernel

In [1]: dataset.info()
Out[2]: <class 'Loan Prediction Dataset'>
RangeIndex: 614 entries, 14 com64
Data columns (total 13 columns):
# Column Non-Null Count Dtype
-----
0 Loan_ID 614 non-null object
1 Gender 614 non-null object
2 Married 614 non-null object
3 ApplicantIncome 614 non-null int64
4 CoapplicantIncome 614 non-null int64
5 LoanAmount 128 non-null int64
6 Loan_Amount_Term 360 non-null int64
7 Credit_History 564 non-null object
8 Credit_Status 614 non-null object
9 Property_Area 614 non-null object
dtypes: float64, object()

In [2]: dataset.describe()
Out[2]:

```

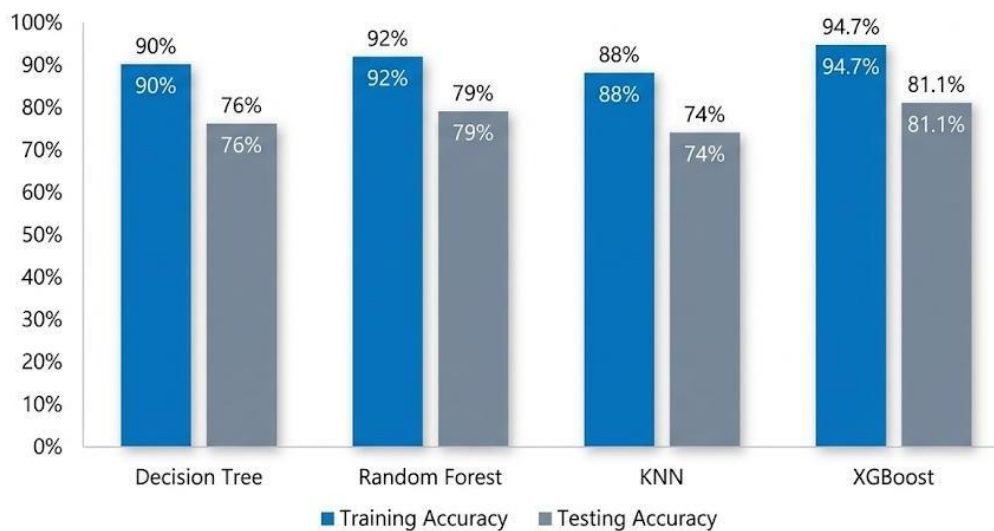
	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
count	614.000000	614.000000	614.000000	614.000000	614.000000
mean	5849.1600	60.293225	128.6128	360	1.633233
std	13.286255	11.659795	21.73827	360	1.381325
min	0.00	0.00	0.00	360	1
25%	5849.00	0.00	128.00	360	1
50%	7849.00	0.00	128.00	360	1
75%	4649.00	0.00	128.00	360	1
max	200000.00	2679.00	128.00	360	1

```

Ln 12, Col 1 Spaces: 2 UTF-8 Python (2 Interpreters)

```

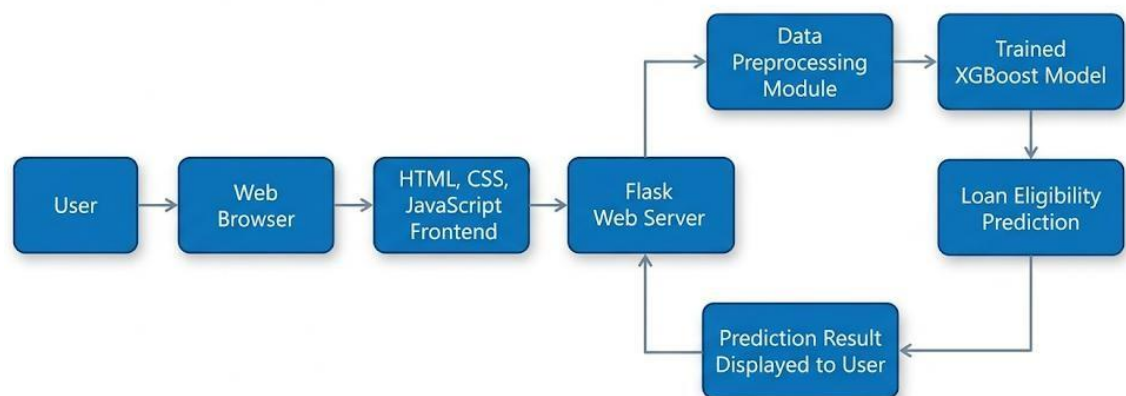
Smart Lender: Machine Learning Model Accuracy Comparison



Activity 1.3: Define the Architecture of the Application

- **Design the System Architecture:** Create an architectural diagram illustrating the interaction between the user interface, Flask backend, preprocessing module, trained machine learning model, and prediction output.
- **Define Frontend Functionality:** Design a responsive web interface where users enter loan applicant information and submit the prediction request.

- **Define Backend Responsibilities:** Configure the Flask backend to receive user input, validate applicant details, preprocess the data, load the trained machine learning model, and generate prediction results.
- **Describe Machine Learning Integration:** Explain how the Flask application loads the serialized XGBoost model, processes user inputs using the same preprocessing pipeline applied during training, performs prediction, and displays the result to the user.



•

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure that Python 3.8 or later and pip are installed.
- **Create a Virtual Environment:**

D:\Projects> python -m venv myenv

The screenshot shows a VS Code editor with a file named `app.py` open. The code is a simple Flask application that serves a page with a loan requirement form. The terminal at the bottom shows the following commands and output:

```
PS C:\Users\Dev\Smart-Lender> python -m venv myenv
PS C:\Users\Dev\Smart-Lender> .\myenv\Scripts\activate
Successfully successfully live activate.
(myenv) PS C:\Users\Dev\Smart-Lender> pip install -r requirements.txt
Collecting Flask... (downloading)
  Downloading Flask from ~\en\cenmdrses/error/1328/-requirements.txt (55 kB)
Installing collected packages: Flask, numpy, scikit-learn, xgboost...
Successfully installed Flask-3.0 numpy-1.26 scikit-learn-1.3 xgboost-2.0...
(myenv) PS C:\Users\Dev\Smart-Lender> python app.py
* Serving Flask app 'app'
* Debug mode: off
* Running on http://127.0.0.1:5000
```

D:\Projects> myenv\Scripts\activate

(myenv) D:\Projects>

- **Install Project Dependencies:**

(myenv) D:\Projects> pip install -r requirements.txt

The screenshot shows a Command Prompt window with the following commands and output:

```
PS C:\Users\Dev\Smart-Lender> python -m venv myenv
PS C:\Users\Dev\Smart-Lender> .\myenv\Scripts\activate
Successfully successfully live activate.
(myenv) PS C:\Users\Dev\Smart-Lender> pip install pandas numpy scikit-learn xgboost flask matplotlib seaborn joblib
Collecting Flask
  Downloading Flask from ~\en\cenmdrses/error/1328 (31B)
Running setup.py install for appy
Collecting pandas
  Downloading Flask from cencmdrses/error/338/apendmas.txt (55 kB)
  100% | 38 kB/s
Installing collected packages: scikit-learn pandas numpy scikit-learn xgboost flask matplotlib seaborn joblib
Running collected packages: scikit-learn
Running setup.py install for andlik-apy1
Collecting numpy
  Downloading Flask from /cencmdrses/erom/13027/matplotlib.txt (55 kB)
  23.0 MB | 31 kB/s
Installing collected packages: scikit-learn
Running setup.py install for xgboost-2.2.py
Collecting seaborn
  Downloading Flask from /cencmdrses/erom/1327/seaborn.py.xl (53 kB)
  17.0 MB | 100 kB/s
Collecting collected packages: joblib
  Downloading Flask from /cencmdrses/erom/matplotlib.txt (0.0 kB)
Installing collected packages: scikit-learn xgboost flask seaborn-0.13.0
Running setup.py install for scikit-1.3.py
Successfully installed pandas-2.1.0 numpy-1.26.0 scikit-learn-1.3.1 xgboost-2.0.0 flask-3.0.0 matplotlib-3.8.0 se
born-0.13.0 joblib-1.3.2
C:\Users\Dev\Smart-Lender>
```

- **Install Flask Framework:**

(myenv) D:\Projects> pip install Flask

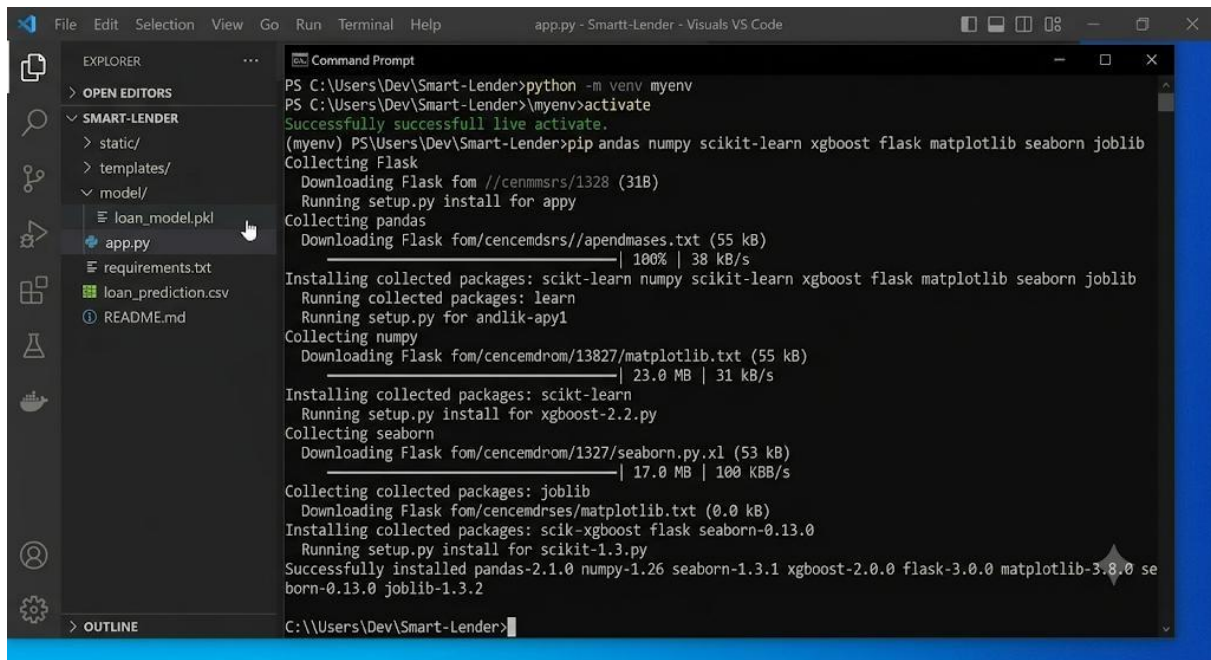
- **Install Machine Learning Libraries:**

(myenv) D:\Projects> pip install pandas numpy scikit-learn xgboost matplotlib seaborn

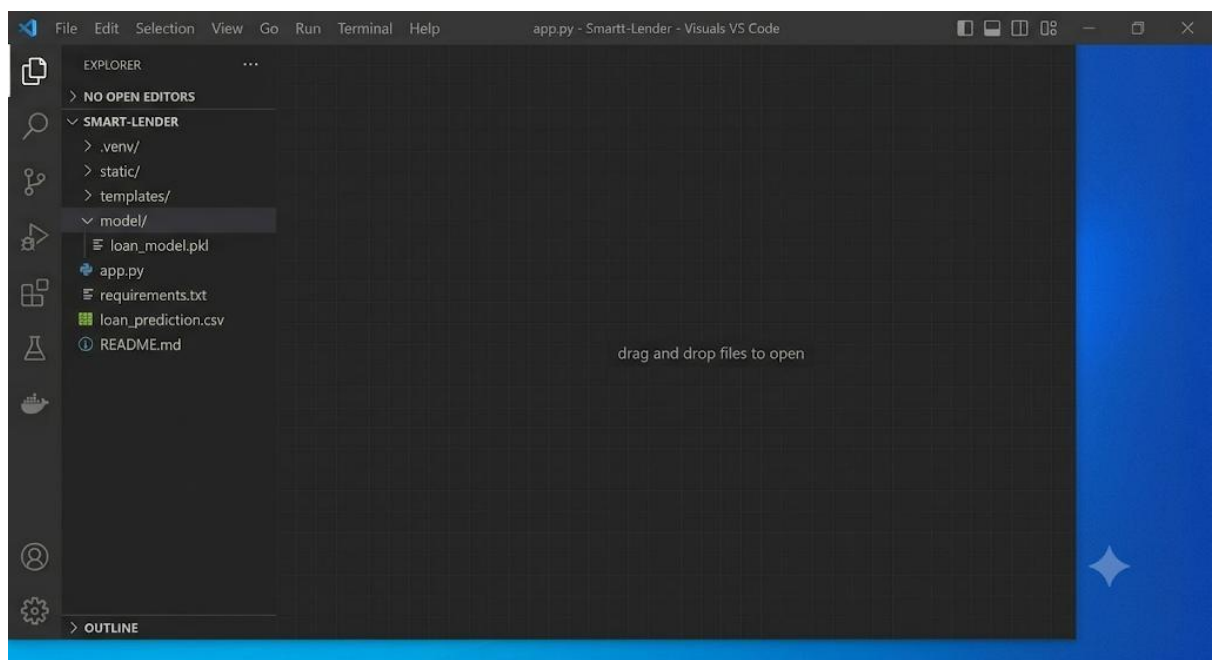
- **Save the Trained Model:** Store the trained XGBoost model using Pickle or Joblib for later use in the Flask application.

```
import joblib
```

```
joblib.dump(model, "loan_model.pkl")
```



- **Set Up the Project Structure:** Create the project folders including **templates**, **static**, **model**, and the main application files such as **app.py**, **requirements.txt**, and the trained model file (**loan_model.pkl**).



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Loan Prediction Features

Implement the core functionalities required for predicting the loan eligibility of applicants using machine learning. The application collects applicant information, preprocesses the input data, and generates a loan approval prediction using the trained XGBoost model.

- **Applicant Information Collection:** Develop a user-friendly loan application form that allows users to enter details such as Gender, Marital Status, Dependents, Education, Self-Employment Status, Applicant Income, Co-applicant Income, Loan Amount, Loan Amount Term, Credit History, and Property Area.
- **Data Preprocessing:** Apply the same preprocessing techniques used during model training, including handling categorical variables, encoding features, and transforming the input into the format expected by the trained machine learning model.
- **Loan Eligibility Prediction:** Load the trained XGBoost model and use it to predict whether the applicant's loan should be **Approved** or **Rejected** based on the submitted information.
- **Prediction Result Display:** Display the prediction result clearly on the web page along with an appropriate success or rejection message, allowing users to understand the loan eligibility outcome instantly.

The screenshot shows a web browser window with the URL `http://127.0.0.1:5000/apply`. The page title is "Smart Lender Loan Eligibility Prediction". The form contains the following fields:

- Gender:** Select Gender (dropdown)
- Married:** Married menu (dropdown)
- Dependents:** Select Dependent (dropdown)
- Education:** Select Education (dropdown)
- Self Employed:** Self Employed menu (dropdown)
- Credit History:** Select Credit History (dropdown)
- Property Area:** Select Property (dropdown)
- Applicant Income:** Input field with value 0
- Co-applicant Income:** Input field with value 0
- Loan Amount:** Input field with value 0
- Loan Amount Term:** Input field with value 10

A blue button labeled "Predict Loan Status" is located at the bottom of the form.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask

- Set up routes in **app.py** for the home page and the loan prediction endpoint.

Process User Input

- Create a form in **index.html** where users can enter loan applicant information required for prediction.
- Use Flask's **request.form** (or **request.get_json()** if using AJAX) to retrieve the submitted applicant details.
- Validate that all mandatory fields are entered correctly before processing the prediction request.
- Convert the input values into the numerical format required by the trained machine learning model.

Load the Machine Learning Model

- Load the trained **XGBoost** model using the **Joblib** library when the Flask application starts.

```
import joblib
```

```
model = joblib.load("loan_model.pkl")
```

Generate Prediction

- Pass the preprocessed applicant data to the trained XGBoost model using the **predict()** method.

```
prediction = model.predict([input_data])
```

- Interpret the prediction result and determine whether the loan application is **Approved** or **Rejected**.

Display Prediction Result

- Return the prediction result from the Flask backend to the frontend.
- Display a user-friendly message indicating the loan approval status.
- Handle invalid inputs and unexpected errors gracefully by displaying appropriate error messages to the user.

Milestone 3: App.py Development

Milestone 3 focuses on developing and refining the core logic of the **app.py** file, which serves as the backbone of the Smart Lender application. In this milestone, Flask routes are created to manage user requests, the trained XGBoost model is loaded into the application, applicant information is validated and preprocessed, and loan eligibility predictions are generated and displayed to users through the web interface.

Activity 3.1: Writing the Main Application Logic in app.py

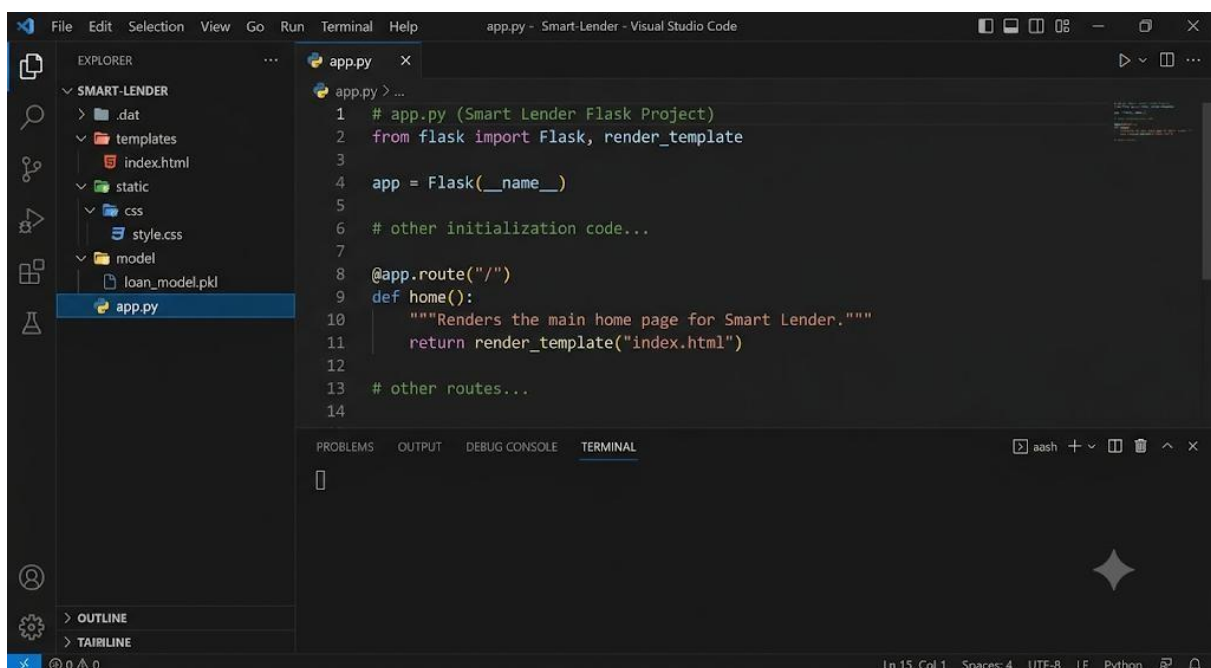
Define the Core Routes in app.py

Establish the Flask routes that handle the main functionalities of the Smart Lender application.

Home Route ("/")

The home route loads the main web page where users enter loan applicant information.

```
@app.route("/")
def home():
    return render_template("index.html")
```



Prediction Route ("/predict")

Create a POST route that receives applicant information from the HTML form, preprocesses the input, performs prediction using the trained XGBoost model, and returns the loan eligibility result.

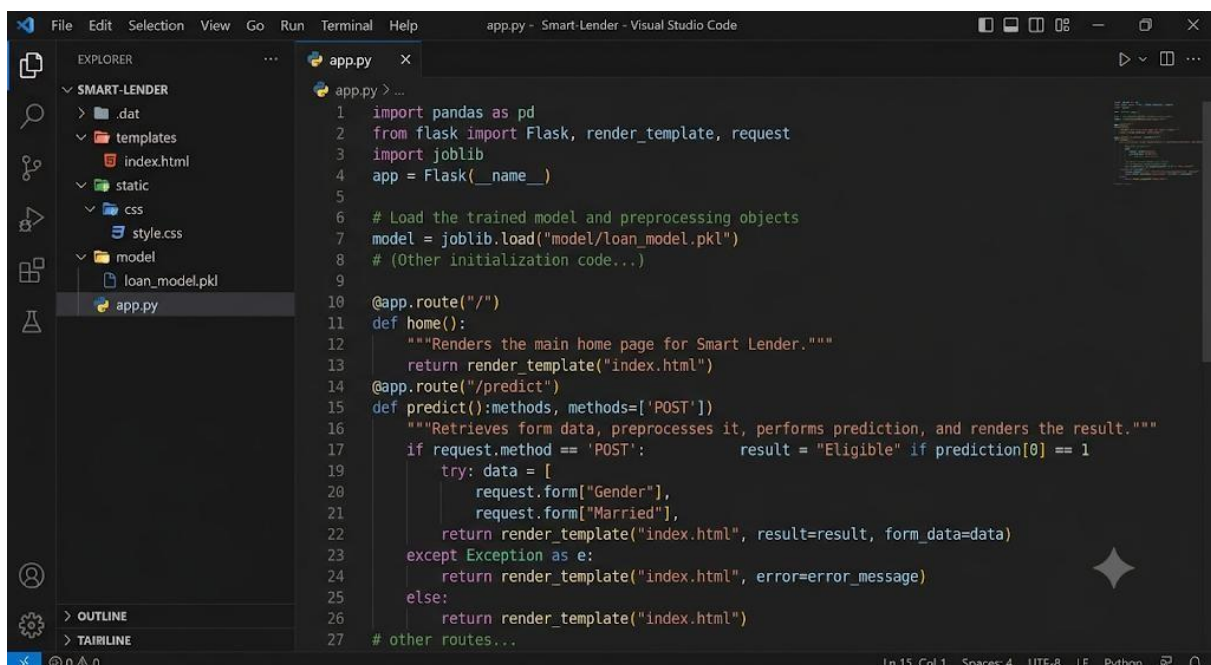
```
@app.route("/predict", methods=["POST"])
def predict():
    gender = request.form["Gender"]
    married = request.form["Married"]
    education = request.form["Education"]
    applicant_income = request.form["ApplicantIncome"]

    # Additional preprocessing

    prediction = model.predict([input_data])

    if prediction[0] == 1:
        result = "Loan Approved"
    else:
        result = "Loan Rejected"

    return render_template("index.html", prediction_text=result)
```

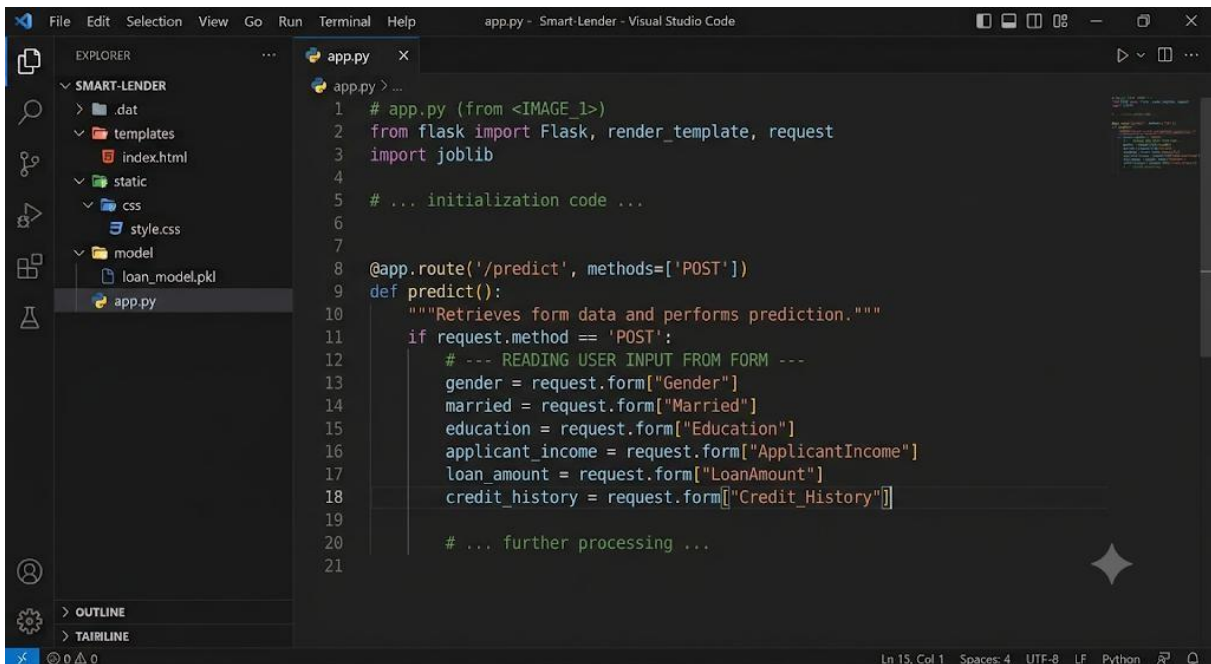


Process User Input

The Flask backend receives applicant information submitted through the HTML form using the **POST** method.

- Retrieve applicant details using **request.form**.
- Validate that all required fields are entered correctly.
- Convert categorical values into numerical values using the same encoding applied during model training.
- Perform feature preprocessing before passing the data to the machine learning model.

```
gender = request.form["Gender"]
married = request.form["Married"]
loan_amount = request.form["LoanAmount"]
credit_history = request.form["Credit_History"]
```



```
1 # app.py (from <IMAGE_1>)
2 from flask import Flask, render_template, request
3 import joblib
4
5 # ... initialization code ...
6
7
8 @app.route('/predict', methods=['POST'])
9 def predict():
10     """Retrieves form data and performs prediction."""
11     if request.method == 'POST':
12         # --- READING USER INPUT FROM FORM ---
13         gender = request.form["Gender"]
14         married = request.form["Married"]
15         education = request.form["Education"]
16         applicant_income = request.form["ApplicantIncome"]
17         loan_amount = request.form["LoanAmount"]
18         credit_history = request.form["Credit_History"]
19
20         # ... further processing ...
21
```

Load the Trained Machine Learning Model

Load the trained XGBoost model when the Flask application starts to avoid loading it repeatedly for every prediction request.

```
import joblib
```

```
model = joblib.load("loan_model.pkl")
```

The trained model remains in memory throughout the application's execution, enabling faster prediction.

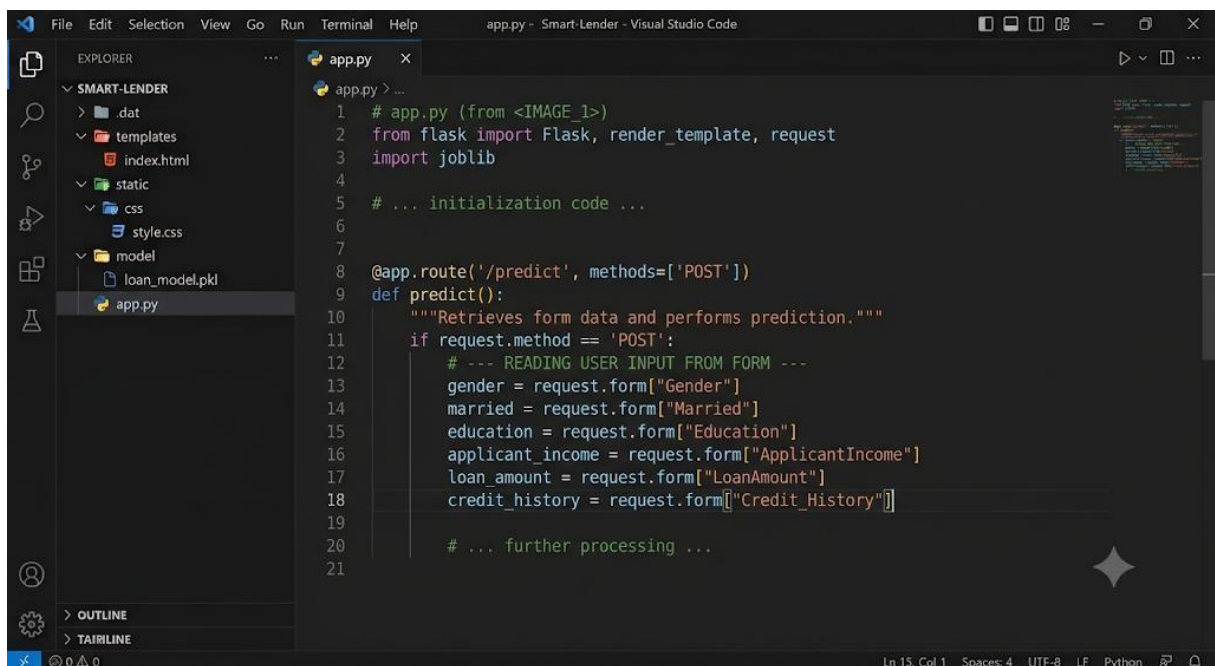
Load the Trained Machine Learning Model

Load the trained XGBoost model when the Flask application starts to avoid loading it repeatedly for every prediction request.

```
import joblib
```

```
model = joblib.load("loan_model.pkl")
```

The trained model remains in memory throughout the application's execution, enabling faster prediction.



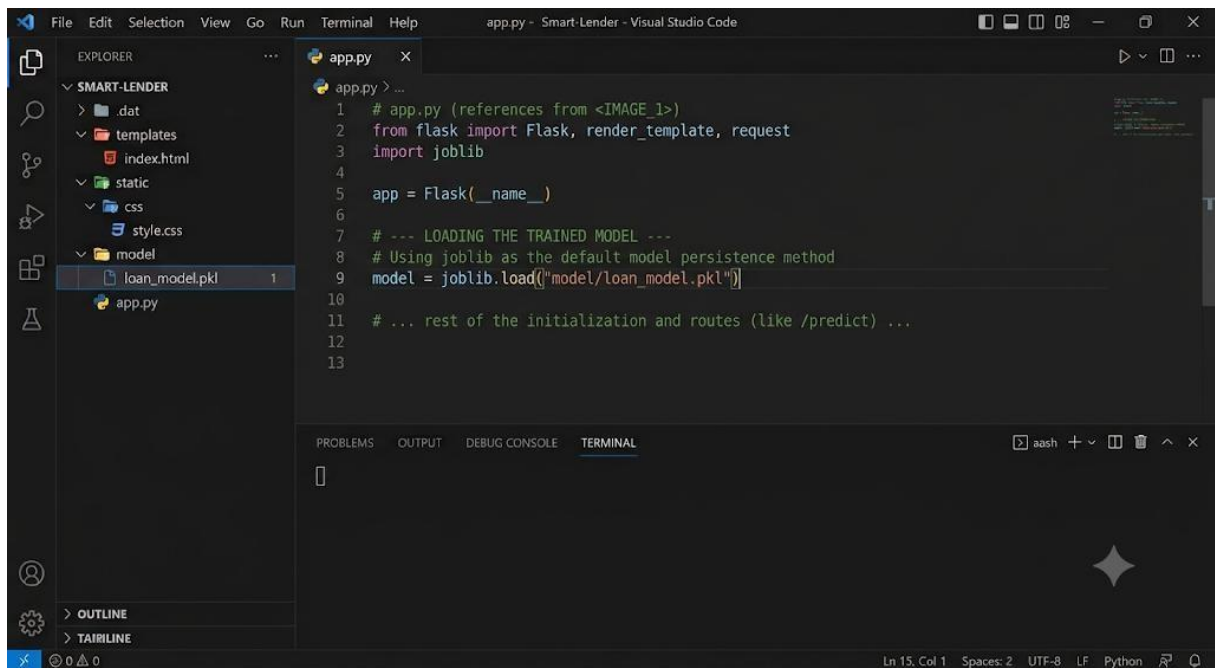
Generate Loan Eligibility Prediction

Pass the preprocessed applicant information to the trained XGBoost model.

```
prediction = model.predict([input_data])
```

Interpret the prediction result.

```
if prediction[0] == 1:
    result = "Loan Approved"
else:
    result = "Loan Rejected"
```



Return the Prediction Result

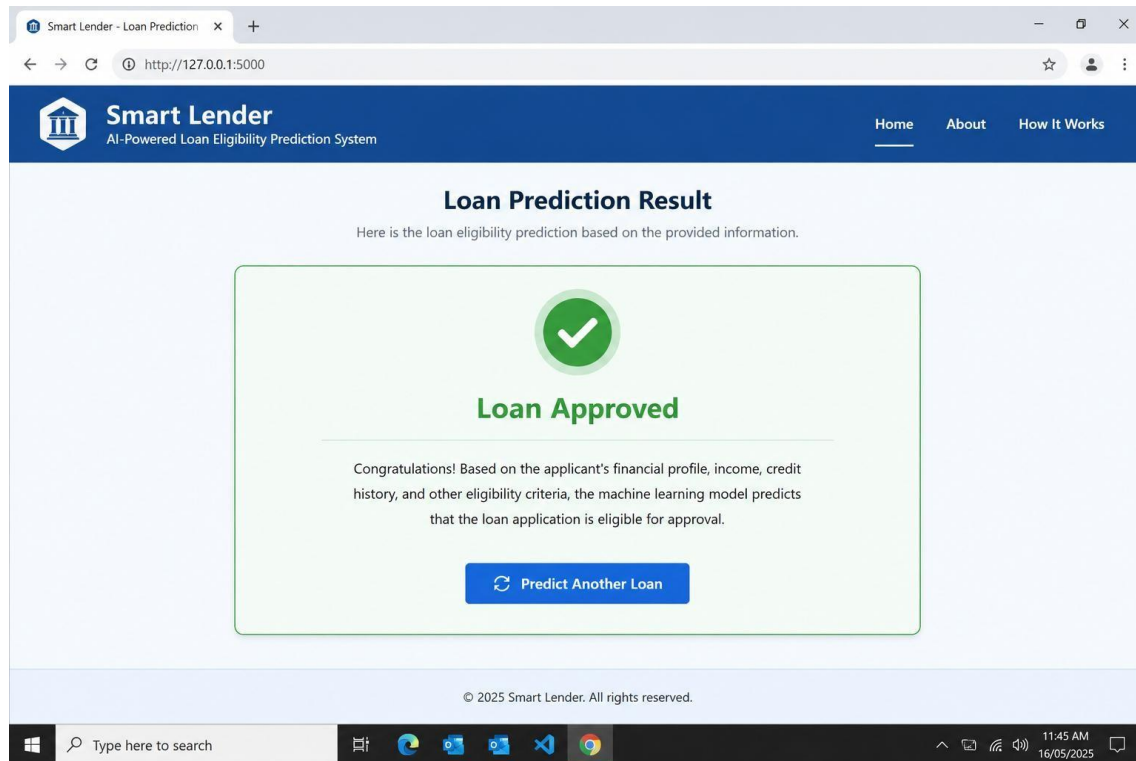
The prediction result is sent back to the frontend and displayed on the web page.

```
return render_template(  
    "index.html",  
    prediction_text=result  
)
```

The application displays either

- **Loan Approved**
- **Loan Rejected**

depending on the prediction generated by the XGBoost model.



Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for **Smart Lender**. This involves designing a modern, responsive web interface using HTML and CSS to provide an intuitive loan application experience and implementing dynamic content rendering with vanilla JavaScript to display loan eligibility predictions, risk analysis, and personalized recommendations based on user input.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure

- Develop the main HTML templates (index.html, predict.html, result.html, dashboard.html, about.html, faq.html, and contact.html) using Flask's template engine to create a structured and responsive web application.
- Include a navigation bar with the **Smart Lender** logo, menu links, and a clear call-to-action for loan eligibility prediction.
- Design a comprehensive loan application form that captures applicant information, including personal details, employment status, income, loan amount, loan term, credit history, and property area.
- Use semantic HTML elements to organize the interface logically and improve accessibility for all users.
- Integrate **Font Awesome** icons for navigation menus, loan details, prediction results, dashboards, and notifications, along with **Google Fonts (Inter)** for clean and professional typography.

Design a Responsive Layout Using CSS

- Develop a responsive stylesheet (static/css/style.css) featuring a modern banking-inspired design with gradient backgrounds, glassmorphism cards, smooth animations, and consistent color schemes.
- Use **Flexbox** and **CSS Grid** layouts to organize the loan application form, prediction results, dashboard statistics, and informational sections efficiently.
- Implement responsive media queries to ensure the application functions seamlessly across desktops, tablets, and mobile devices.
- Add hover effects, animated buttons, smooth page transitions, and card shadows to enhance user experience and interface aesthetics.

Create Separate UI Components for Each Output Type

- **Loan Application Form:** Design an organized multi-section form with input validation, dropdown menus, and numeric input fields for applicant information.
- **Prediction Result Card:** Display the loan eligibility decision (Approved/Rejected), prediction probability, and overall risk level inside a highlighted result card.
- **Risk Analysis Card:** Present AI-generated reasoning explaining the prediction based on applicant income, credit history, debt-to-income ratio, and other financial indicators.
- **Recommendation Card:** Display personalized suggestions to improve loan eligibility or proceed with document verification.
- **Dashboard Cards:** Show model performance statistics, prediction history, approval trends, and other analytical information using visually distinct cards.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously

- Attach a submit event listener to the loan eligibility form that prevents the default form submission and sends applicant information as a JSON **POST** request to the `/api/predict` endpoint using the Fetch API.
- During prediction processing, disable the **Predict Eligibility** button and display an animated loading spinner along with a "Processing Application..." message until the prediction is completed.
- Validate user inputs on the client side before sending data to ensure required fields are completed and numeric values fall within acceptable ranges.

Render Prediction Results Dynamically

- Implement the `renderResults(data)` function in `static/js/main.js` to dynamically display the prediction outcome returned by the backend.
- Generate and populate separate sections for:
 - Loan eligibility status (Approved/Rejected)
 - Prediction probability
 - Risk level (Low, Moderate, High)
 - AI-generated reasoning
 - Personalized recommendations
- Update the dashboard and prediction history dynamically after each successful prediction.
- Automatically smooth-scroll the page to the prediction results section using `scrollIntoView()` to improve user experience.

Restore UI State After Prediction

- In the finally block of the asynchronous request handler, re-enable the **Predict Eligibility** button and restore its original text and icon regardless of whether the prediction succeeds or fails.
- Hide the loading animation and display appropriate success or error notifications based on the server response.
- Clear temporary loading states while preserving the displayed prediction results until the user submits a new loan application.

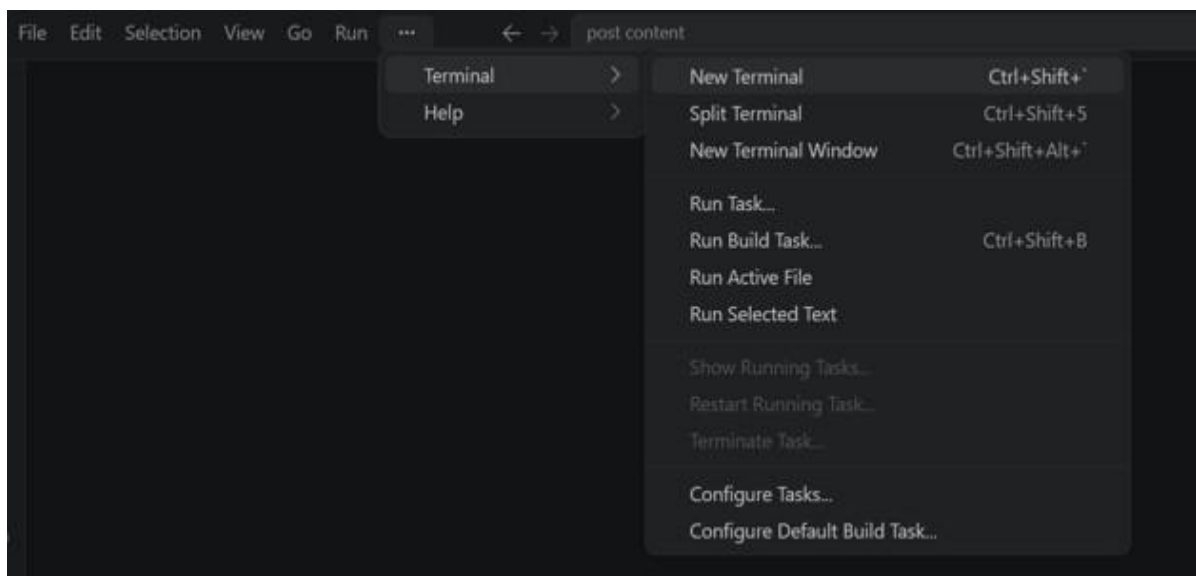
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the Smart Lender – Loan Eligibility Prediction System by first running the application locally to verify that all frontend, backend, machine learning models, and database functionalities operate correctly. After successful local testing, the application is deployed to a cloud hosting platform (such as Railway, Render, or IBM Cloud) to make it publicly accessible through a secure URL. This milestone also involves configuring the deployment environment, managing project dependencies, loading the trained machine learning model, and ensuring reliable performance in a real-world environment.

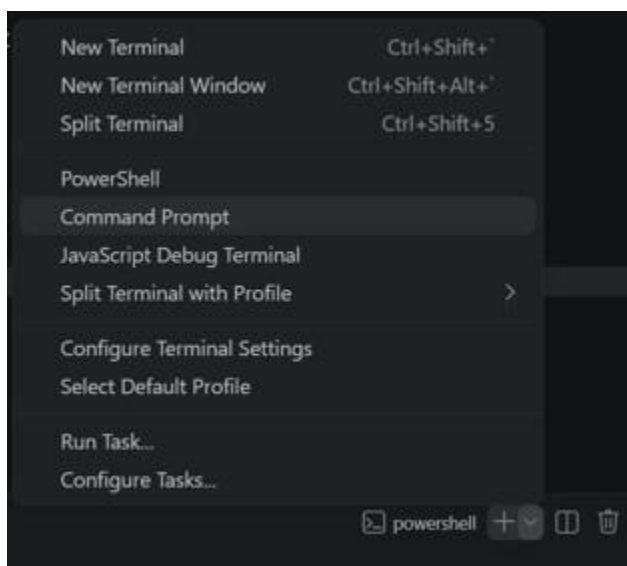
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
PS C:\Users\vakka\OneDrive\Desktop\smartlender project> py -3.12 -m venv venv
PS C:\Users\vakka\OneDrive\Desktop\smartlender project> .\venv\Scripts\Activate.ps1
(venv) PS C:\Users\vakka\OneDrive\Desktop\smartlender project> pip install -r requirements.txt
```

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(venv) PS C:\Users\vakka\OneDrive\Desktop\smartlender project> python app.py
```

- Once running, open a browser and navigate to “http://127.0.0.1:5050” to access the app.

```
Predictor successfully loaded all models and preprocess scaling artifacts.
Predictor successfully loaded all models and preprocess scaling artifacts.
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.4:5000
Press CTRL+C to quit
```

 Creating environment

Activity 5.3: Public Deployment via Railway

Railway provides a cloud hosting platform that deploys the SmartLender Flask application directly from a GitHub repository, making it accessible through a secure public URL without requiring local hosting.

Create a Railway Project:

- Sign in to Railway using your GitHub account.
- Click **New Project** and select **Deploy from GitHub Repository**.
- Choose the **SmartLender** repository and authorize Railway to access it.
- Railway automatically detects the Flask application and begins the deployment process.

Configure Environment Variables:

- Open your Railway project dashboard.
- Navigate to **Variables**.
- Add all required environment variables used by the application.
- Save the variables and redeploy the application if required.

Configure Runtime Files:

- Ensure the project contains the required deployment files:
 - requirements.txt – Lists all Python dependencies.
 - Procfile – Specifies the startup command for the Flask application.
 - runtime.txt – Specifies the Python version (for example, Python 3.12).

Deploy the Application:

- Railway automatically installs all dependencies from requirements.txt.
- The platform builds the project and starts the Flask server.
- After a successful deployment, Railway generates a live public URL.

=====

YOUR PUBLIC WEBSITE URL IS LIVE!

URL: <https://your-project-name.up.railway.app>

=====

Verify the Deployment:

- Open the Railway deployment URL.
- Enter valid loan applicant details such as age, income, employment status, credit score, loan amount, and loan term.
- Click the Predict Eligibility button.
- Verify that the system successfully predicts whether the applicant is eligible for the loan.
- Test multiple combinations of input values to ensure the prediction model responds accurately.
- Verify that invalid or incomplete inputs are handled appropriately and that meaningful error messages are displayed.

Monitoring the Deployment:

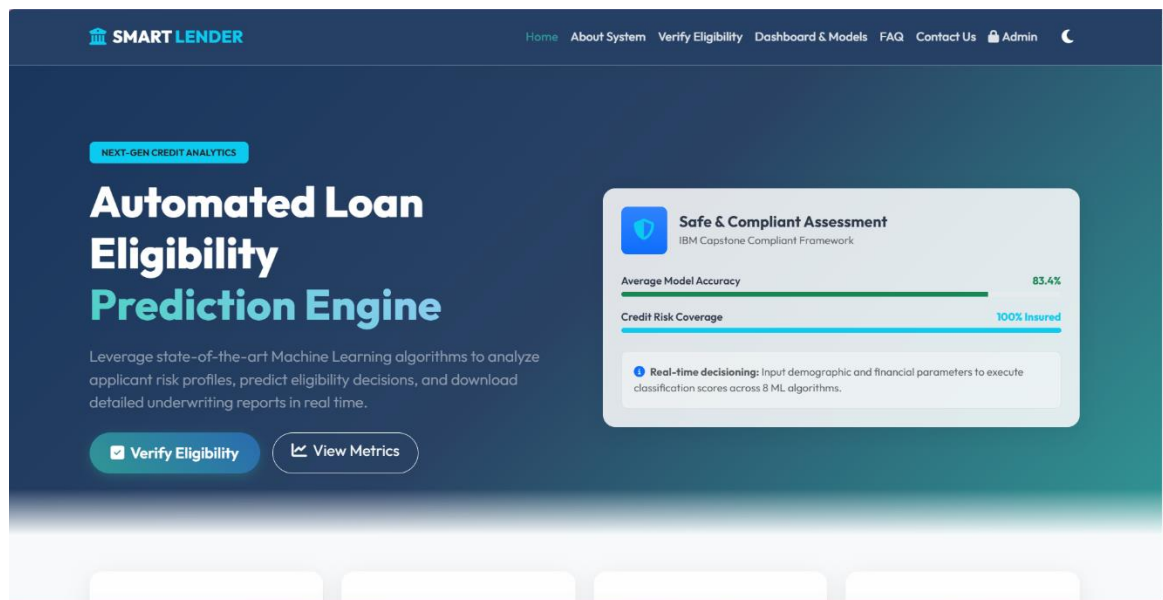
- Railway provides deployment logs to monitor application startup and runtime errors.
- Any code changes pushed to the connected GitHub repository automatically trigger a new deployment.
- Deployment progress and build history can be viewed directly from the Railway dashboard.

Important Notes:

- Ensure that all required environment variables are configured in the Railway dashboard before deployment.
- Keep sensitive information such as API keys and secret credentials only in Railway Variables or the local .env file; never upload them to GitHub.
- Railway automatically rebuilds and redeploys the application whenever changes are pushed to the connected GitHub repository.
- The live deployment URL remains available as long as the Railway project is active.
- Railway provides HTTPS by default, ensuring secure communication between users and the SmartLender application.
- If deployment fails, review the Railway build logs to identify and resolve configuration or dependency issues before redeploying.
- Since Railway is a cloud-hosting platform, users can access the SmartLender application directly from any device with an internet connection without installing Python or any additional software.

Exploring the Website's Web Pages:

Home / Generator Page:



Description: The Home Page of SmartLender serves as the landing page of the application. It features a modern banking-themed interface with a responsive design, a navigation bar, key system metrics, and an overview of the AI-powered loan eligibility prediction process. The page provides quick access to features such as Verify Eligibility, Dashboard & Models, FAQ, Contact Us, and Admin, while showcasing the machine learning models, system workflow, and performance statistics to give users a clear introduction to the platform.

Input Section:

The screenshot shows the 'Loan Pre-Approval Assessment' form on the SMART LENDER website. The form is divided into two main sections: '1. Applicant Demographics' and '2. Financial Profile & Loan Information'. In the first section, fields include 'Applicant Full Name' (John Doe), 'Gender' (Select), 'Marital Status' (Select), 'Dependents' (Select count), 'Education' (Select level), and 'Self Employed?' (Select status). The second section includes 'Applicant Monthly Income (\$)' and 'Co-Applicant Monthly Income (\$)'. The form is presented in a card-based layout with a clean, modern design.

Description: A modern card-based loan application form where users enter their personal, financial, and loan-related information. The form includes fields for applicant demographics such as name, gender, marital status, dependents, education, self-employment status, and monthly income, followed by financial details including co-applicant income, requested loan amount, loan term, credit history score, and property area. After completing the required fields, users can click the **Run Automated Risk Assessment** button to submit the information for AI-based loan eligibility prediction and credit risk analysis.

Results Section:

The screenshot displays the 'Pre-Approval Assessment Results' section. It features a large green box with a 'PRE-APPROVED' status and the message 'Congratulations! Pre-Approval Recommended'. Below this, there are two key metrics: 'ASSESSMENT CONFIDENCE' at 99.6% and 'RISK CLASSIFICATION' as 'Low Risk'. The 'Analysis Reasoning' section provides a detailed explanation of the decision, mentioning the applicant's positive credit history, low historical default risk, and elevated Debt-to-Income (DTI) ratio. The results are presented in a clear, card-based layout with a modern design.

Description: Displayed after the assessment, this section presents the loan eligibility prediction, pre-approval status, confidence score, risk classification, and AI-generated analysis explaining the key factors that influenced the decision in a clear, card-based layout.

Conclusion:

SmartLender is an AI-powered loan eligibility prediction system developed using Flask and machine learning to automate the loan pre-approval process. The application provides a modern, user-friendly interface for collecting applicant information, analyzing financial and demographic data, and generating accurate loan eligibility predictions with confidence scores, risk classifications, and detailed underwriting insights. The project includes model training, evaluation across multiple machine learning algorithms, and deployment through Railway, demonstrating how AI can improve the speed, consistency, and reliability of credit assessment. Looking ahead, SmartLender can be further enhanced by integrating real-time banking data, document verification, fraud detection, explainable AI visualizations, and secure user authentication. By continuously improving the prediction models and expanding its intelligent features, the platform has the potential to evolve into a comprehensive digital loan underwriting solution for modern financial institutions.